

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

Copy as Markdown

Deferrable constraints

The database checks a foreign key at the end of every statement. Insert a child row before its parent exists and that statement fails, even when the parent is inserted a line later in the same transaction. Marking the relation as deferrable moves the check to the end of the transaction, so the rows only need to line up by the time it commits.

This is what makes circular references between two tables writable: with a per-statement check, neither row can be written first.

Declare a deferrable relation

Add `deferrable` or `deferred` to the relation on the side that holds the foreign key:

```
# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  companyId: int, relation(parent=company, deferrable)
```

Keyword	Constraint	Checked
<code>deferrable</code>	<code>DEFERRABLE INITIALLY IMMEDIATE</code>	After each statement, unless the transaction defers it.
<code>deferred</code>	<code>DEFERRABLE INITIALLY DEFERRED</code>	At commit.



The two keywords are mutually exclusive. A relation declared with neither keyword cannot be deferred at runtime.

Both work on [id relations](#) and [object relations](#). Like `onUpdate` and `onDelete`, they can only be set on the side holding the foreign key:

```
# employee.spy.yaml
class: Employee
table: employee
fields:
  name: String
  company: Company?, relation(deferred)
```

Adding, changing, or removing the keyword changes the constraint in the database, so run `serverpod create-migration` afterwards.

Defer the check in a transaction

A `deferred` relation is checked at commit already, so ordinary transactions can write the rows in any order:

```
await session.db.transaction((transaction) async {
  await Employee.db.insertRow(session, employee, transaction: transaction);
  await Company.db.insertRow(session, company, transaction: transaction);
});
```

A `deferrable` relation keeps the per-statement check until a transaction asks for it to be deferred. Set `deferConstraints` in the [transaction settings](#) to defer every deferrable constraint for the duration of the transaction:

```
await session.db.transaction(
  (transaction) async {
    await Employee.db.insertRow(session, employee, transaction: transaction);
    await Company.db.insertRow(session, company, transaction: transaction);
  },
  settings: TransactionSettings(deferConstraints: true),
);
```

Either way the constraint is still enforced. If the parent row is missing when the transaction commits, the commit fails with a `DatabaseQueryException` and the whole transaction is rolled back.

NOTE

On SQLite the `deferConstraints` setting defers every foreign key check in the transaction, whether or not the relation is declared deferrable. On PostgreSQL only constraints declared with `deferrable`

or **deferred** can be deferred.

Related

- [Referential actions](#): what happens to related rows when the row they point to is deleted or its id changes.
- [Transactions](#): isolation levels, savepoints, and rollback.